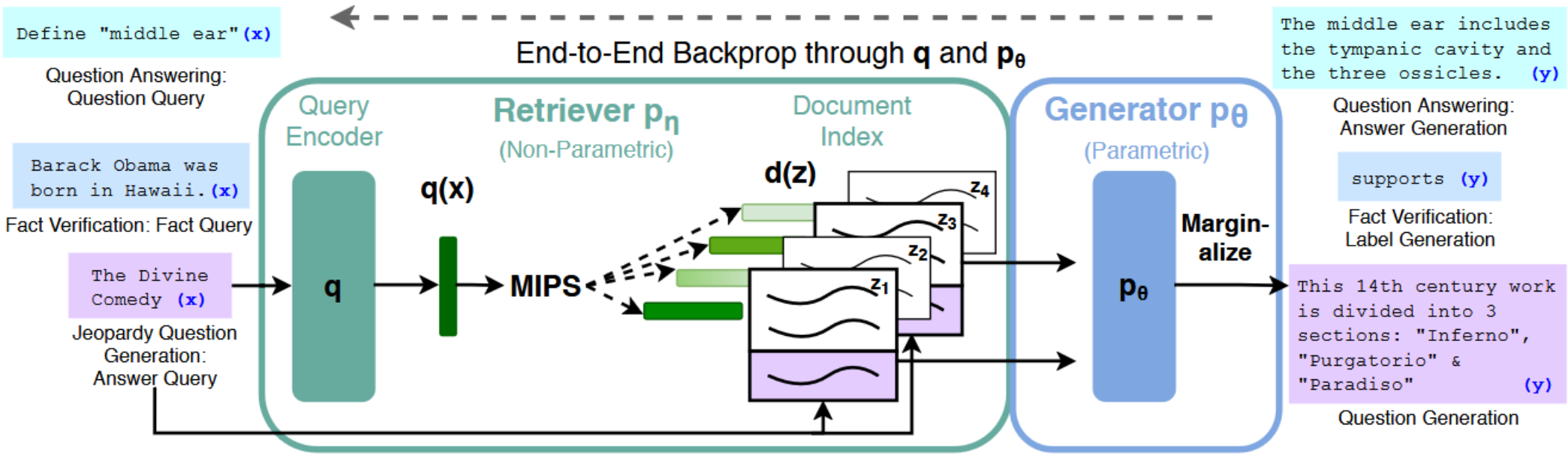
UNIVERSIDAD EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

Inteligencia Artificial

Lecture12 – Retrieval Augmented Generation



Agenda

1. Modelos de Lenguaje
2. RAG
3. Embeddings

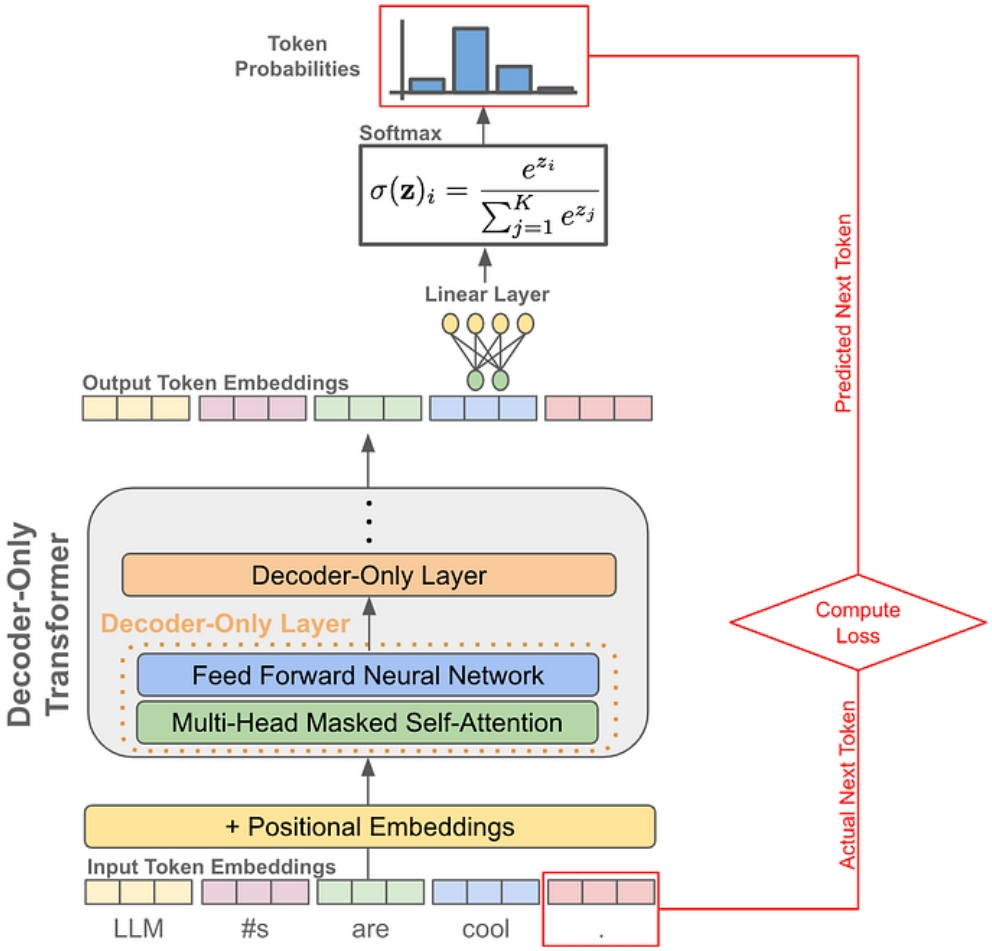
LLMs

Un modelo de lenguaje es un modelo probabilístico que asigna probabilidades a secuencias de palabras.

En la práctica: Dado un texto de entrada (prompt), el modelo predice cuál es el siguiente token más probable — y repite este proceso para generar la respuesta completa.

Entrenamiento
Se entrena en enormes corpus de texto (Wikipedia, libros, web). El modelo aprende patrones estadísticos del lenguaje.

Inferencia
Dado un prompt, genera tokens uno a uno hasta completar la respuesta. Es autoregresivo — cada token depende de los anteriores.

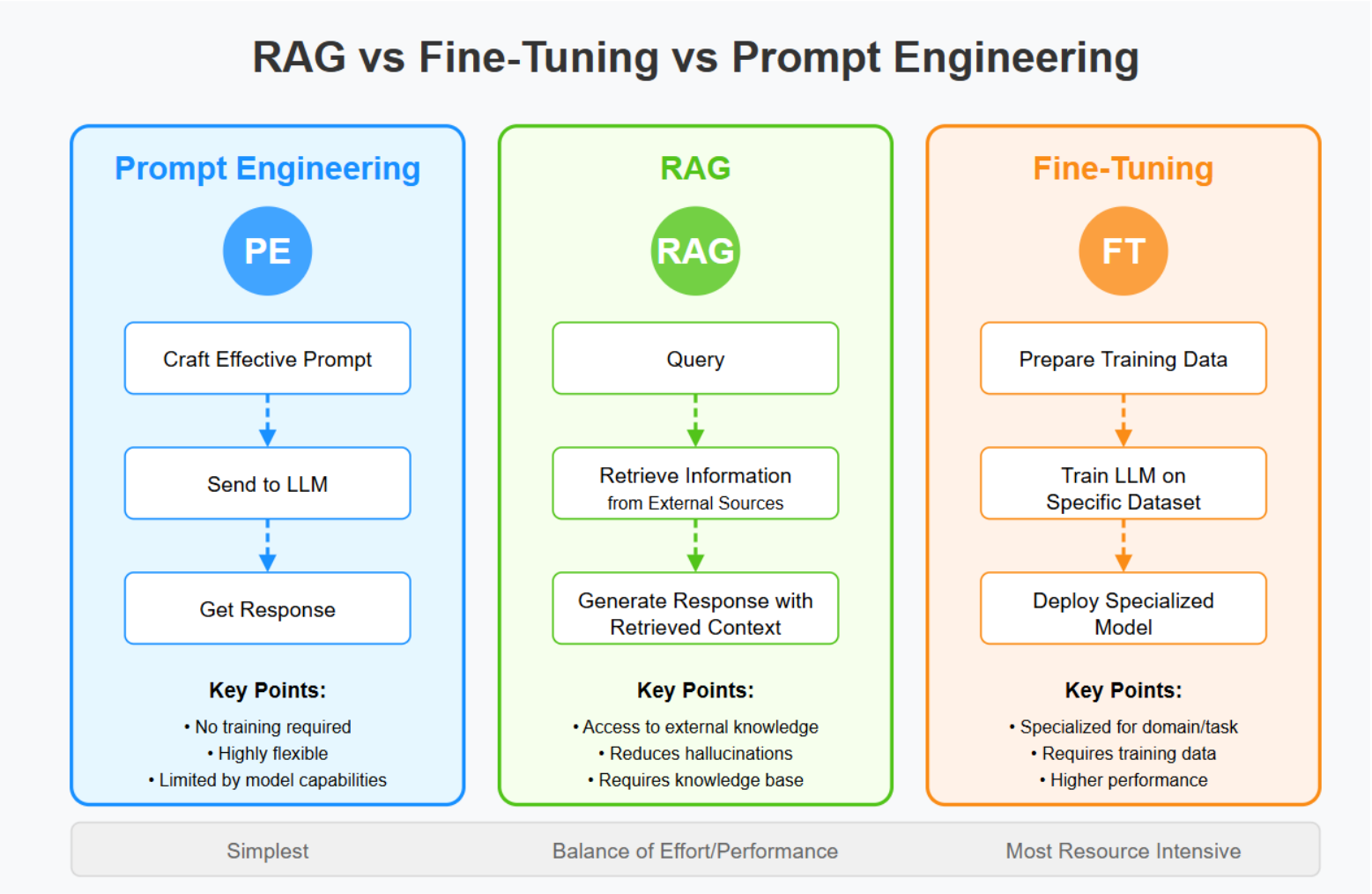


Problema - Contexto

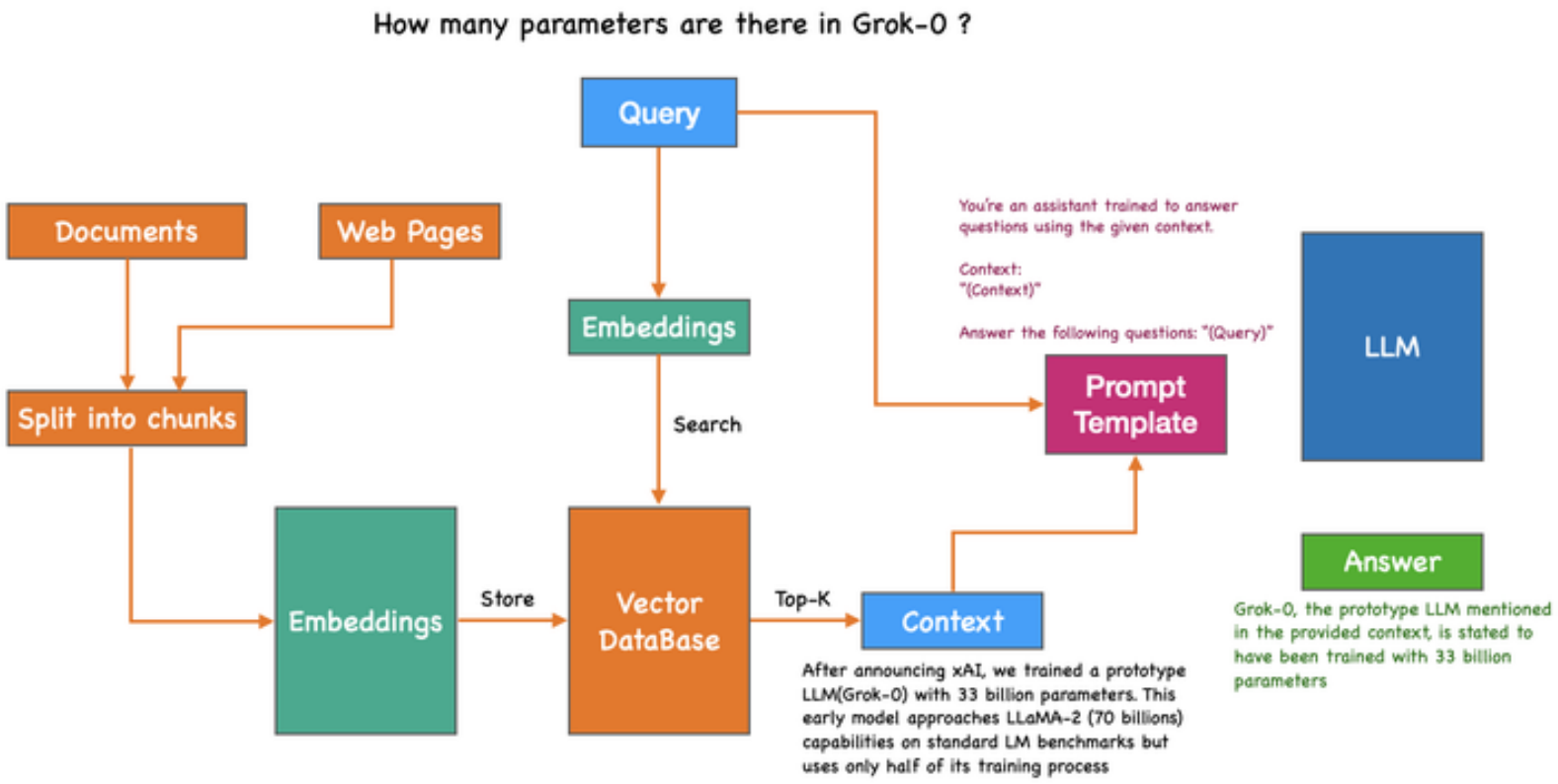
UNIVERSIDAD EAFIT | Escuela de Ciencias Aplicadas e Ingeniería

El LLM solo sabe lo que vio en el entrenamiento:

- No conoce tus documentos internos
- No conoce eventos recientes (tiene fecha de corte)
- No puede acceder a datos en tiempo real
- Puede "alucinar" — inventar información plausible pero falsa



Retrieval Augmented Generation - RAG



Desde primeros principios: RAG es una función donde, dada una pregunta del usuario, la convertimos en vectores, buscamos los vectores más similares en nuestra base de datos, y concatenamos los chunks recuperados al prompt. Así de simple.

¿Por qué Chunking?

UNIVERSIDAD EAFIT | Escuela de Ciencias Aplicadas e Ingeniería

No podemos meter documentos completos en cada query — hay dos razones principales.

Ventana de contexto
Los LLMs tienen un límite de tokens por request. Un documento de 100 páginas no cabe. Los chunks resuelven esto.

Calidad del retrieval
Un chunk que habla de un solo tema genera un vector más representativo. Documentos largos mezclan temas y degradan la búsqueda.

Tamaño	Problema
Muy pequeño	Falta contexto, chunks sin sentido completo
Muy grande	Vectores diluidos, retrieval impreciso
~400-600 chars	Balance óptimo para la mayoría de casos

Chunking semántico respeta párrafos y oraciones — produce chunks con significado cohesivo.

Tipos de Búsqueda

UNIVERSIDAD EAFIT | Escuela de Ciencias Aplicadas e Ingeniería

Vectorial
Busca por similitud semántica. Entiende sinónimos y contexto.

⚠ Puede devolver resultados relacionados pero no exactos

Keyword (BM25)
Coincidencias exactas de términos. Ideal para nombres propios y siglas.

⚠ No entiende relaciones semánticas

Híbrida ← producción
Combina vectorial + keyword. Reordena por scores combinados.

✅ Mejor recall y precision en conjunto

Ejemplo: buscar "política de vacaciones" con búsqueda vectorial puede devolver "días de descanso" (semánticamente similar pero sin la palabra exacta). BM25 lo perdería. Híbrida encuentra ambos.

Embeddings

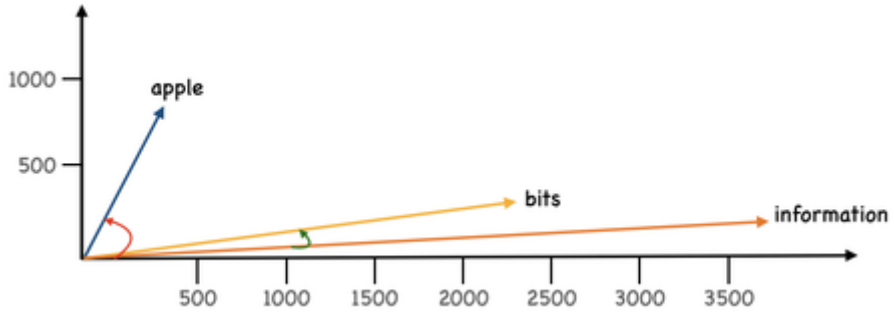
Un embedding es una función que convierte texto en un vector de números reales. Textos semánticamente similares quedan **cerca en el espacio vectorial.**

Hipótesis distribucional: palabras que aparecen en el mismo contexto tienden a tener significados similares. "teacher" y "professor" suelen aparecer junto a "school", "exam", "course".

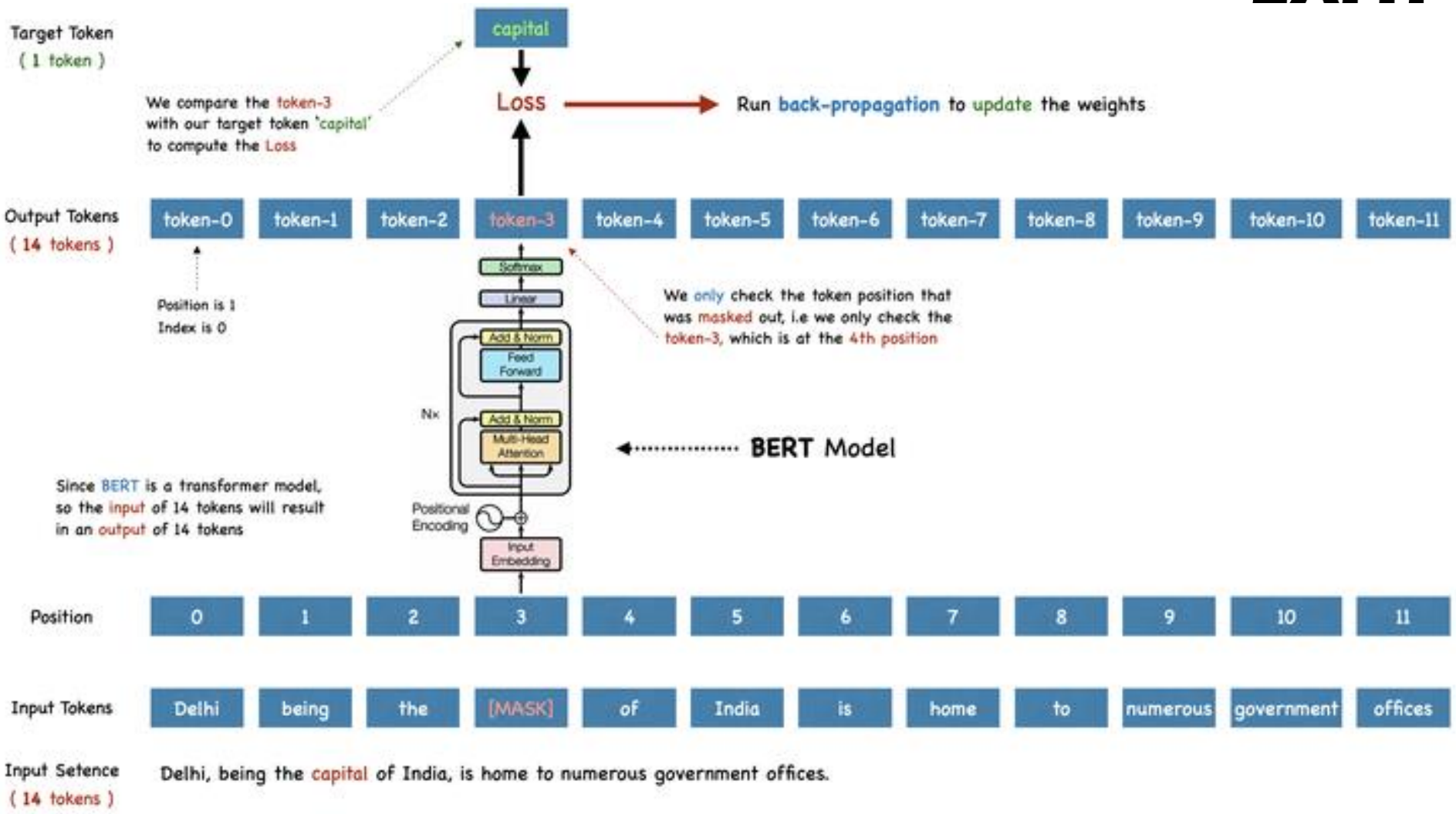
Self-Attention en Transformers captura exactamente esto — relaciona cada token con todos los demás de la oración.

Similitud del coseno

```
cos(A,B) = dot(A,B) / (‖A‖·‖B‖) # Rango: -1 (opuestos)
→ 1 (idénticos) # 0 = ortogonales (sin relación)
sim("gato", "felino") → 0.89 sim("gato", "reembolso") →
0.12
```

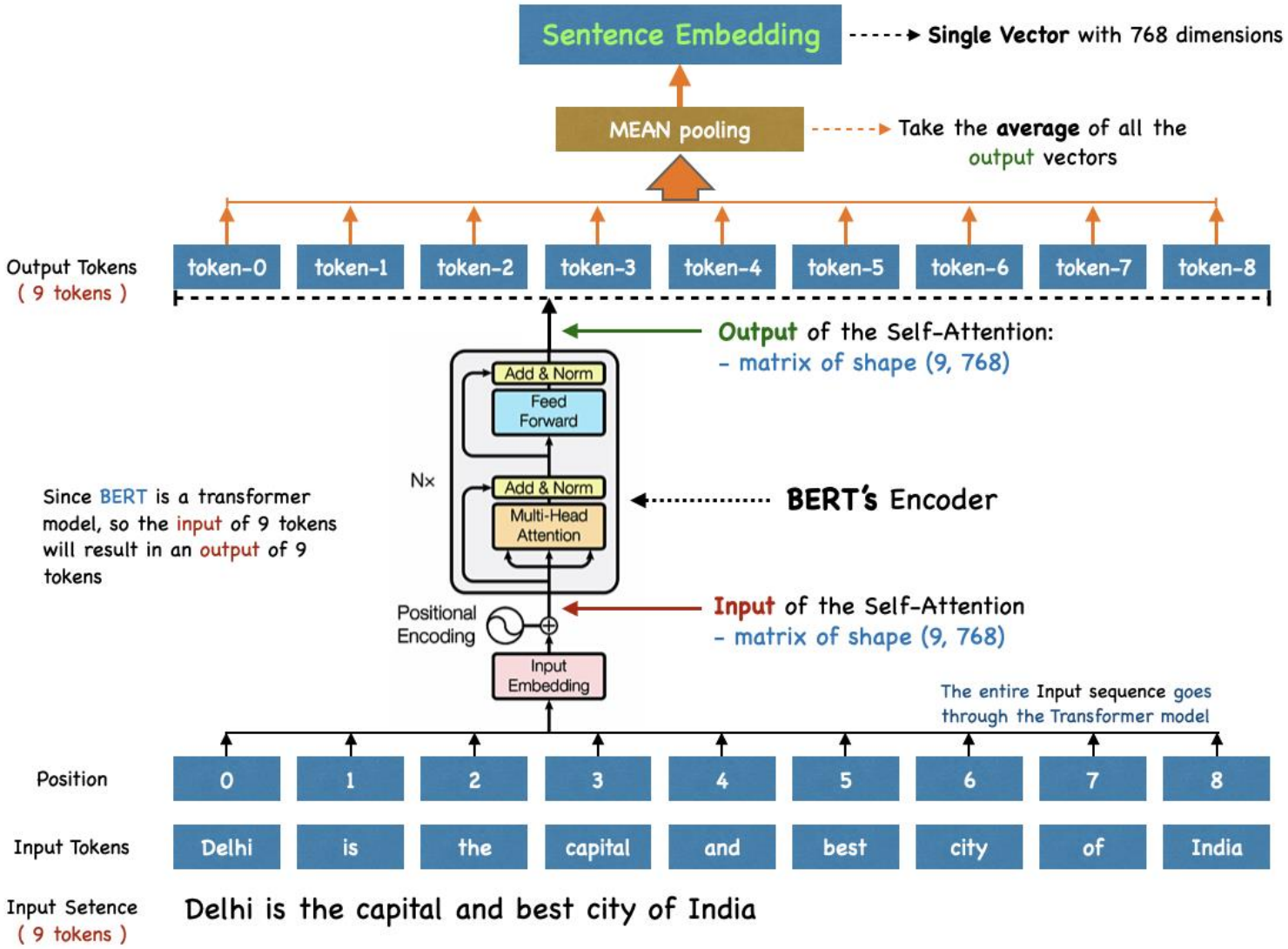


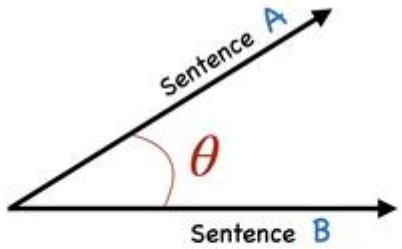
BERT



Self-Attention captura el "significado" de una oración completa.

Embeddings de Oraciones





$$cosine\ similarity = S_C(A,B) = cos(\theta) = \frac{\mathbf{A}\,.\,\mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Sentence-BERT

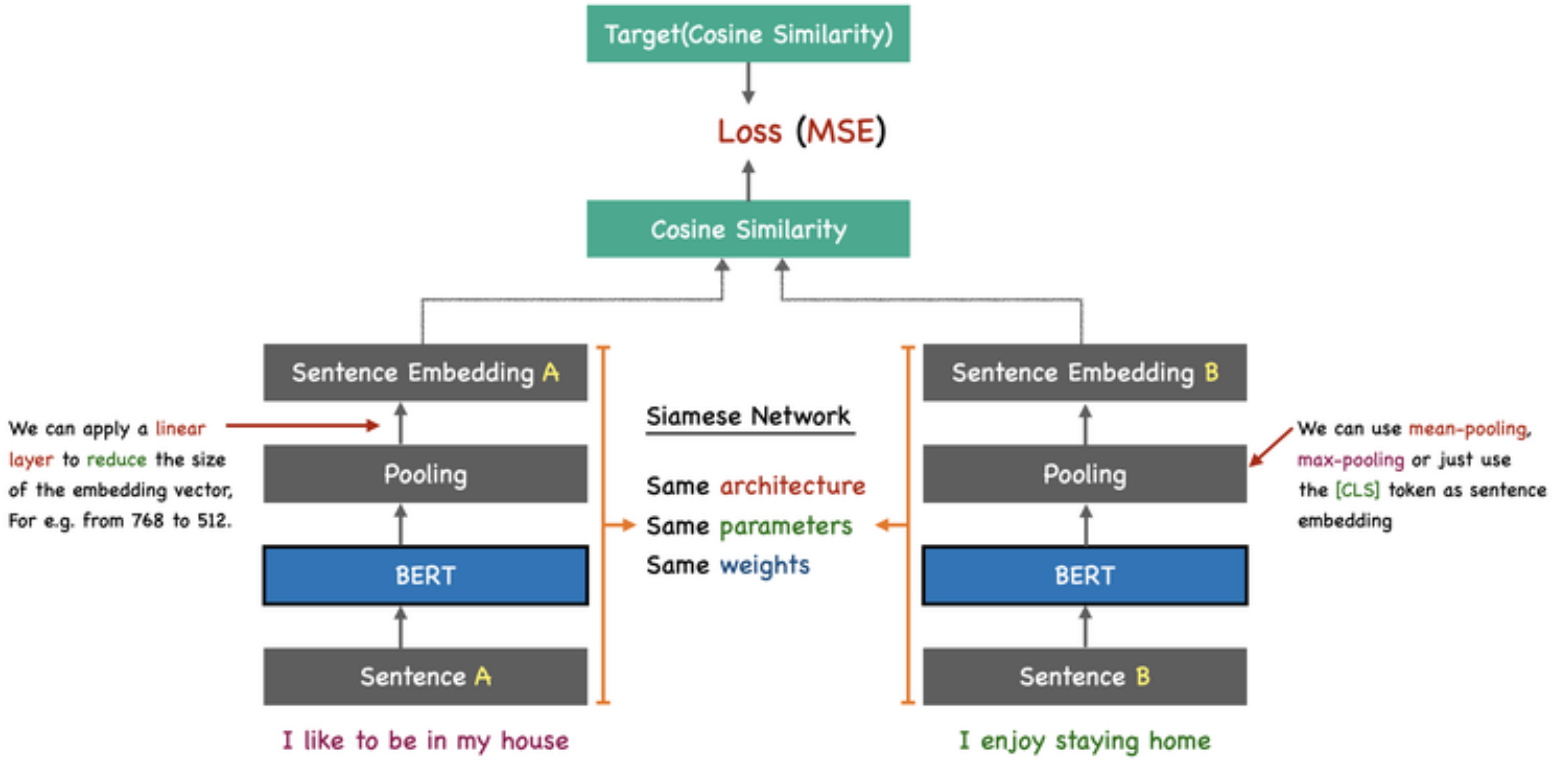
Problema: Nadie le dijo a BERT que los embeddings que produce deberían ser comparables con la similitud del coseno, es decir, que dos oraciones similares deberían estar representadas por vectores que apunten en la misma dirección en el espacio.

Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks

Nils Reimers and Iryna Gurevych
Ubiquitous Knowledge Processing Lab (UKP-TUDA)
Department of Computer Science, Technische Universität Darmstadt
www.ukp.tu-darmstadt.de

Sentence A *- I like to be in my house*

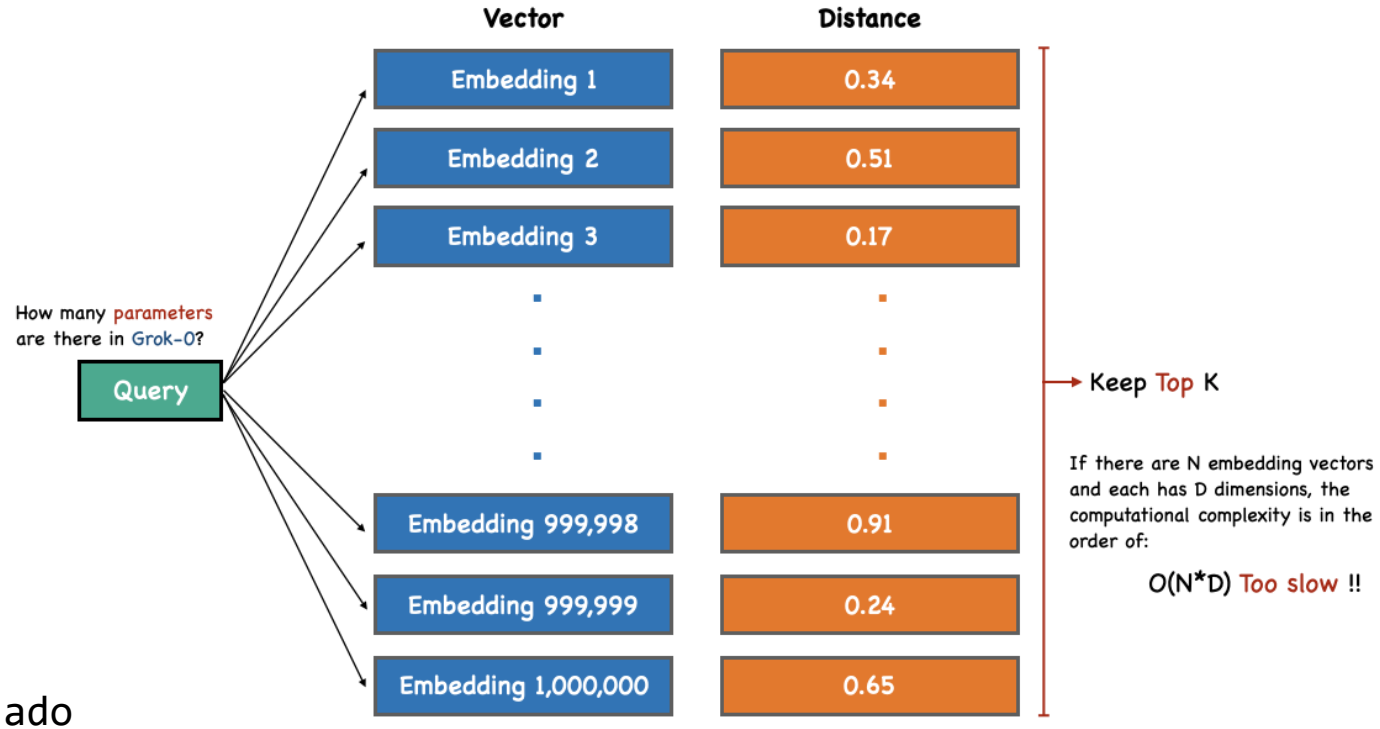
Sentence B *- I enjoy staying home*



Base de datos vectorial

Vector DB: almacena embeddings (vectores de dimensión fija)

- **Objetivo:** encontrar elementos similares
- **Similitud:** coseno o distancia euclidiana
- **Método:** KNN / búsqueda aproximada (ANN)
- **Uso:** recomendación y búsqueda
- **Valor:** recuperación semántica rápida a escala



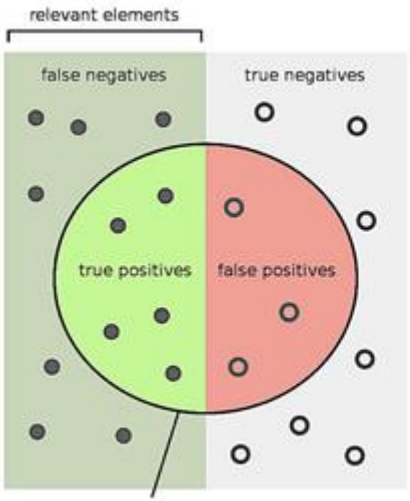
Con 1M de chunks de 768 dims:
768 millones de operaciones por query. Demasiado lento para producción.

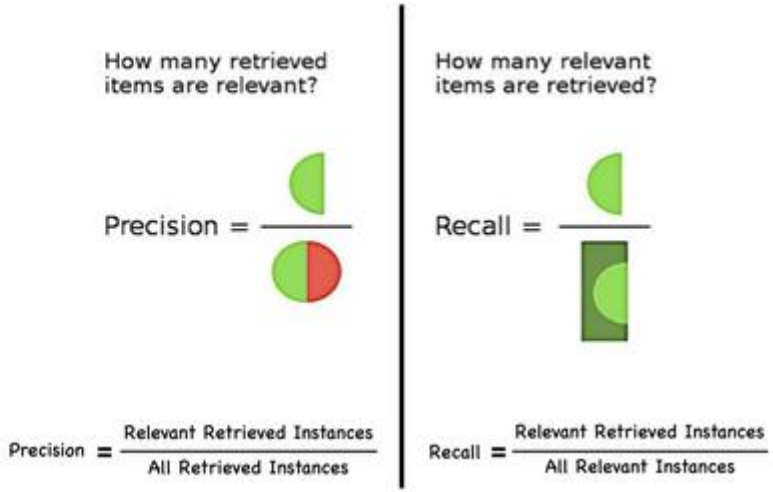
ANN: Cambiar precisión por velocidad

La idea: reducir el número de comparaciones pero obtener resultados precisos con **alta probabilidad.**

Métricas que importan:

Métrica	**Qué mide**
Recall	¿Qué % de los K vecinos reales encontramos?
Latencia	Tiempo de respuesta por query
Throughput	Queries por segundo

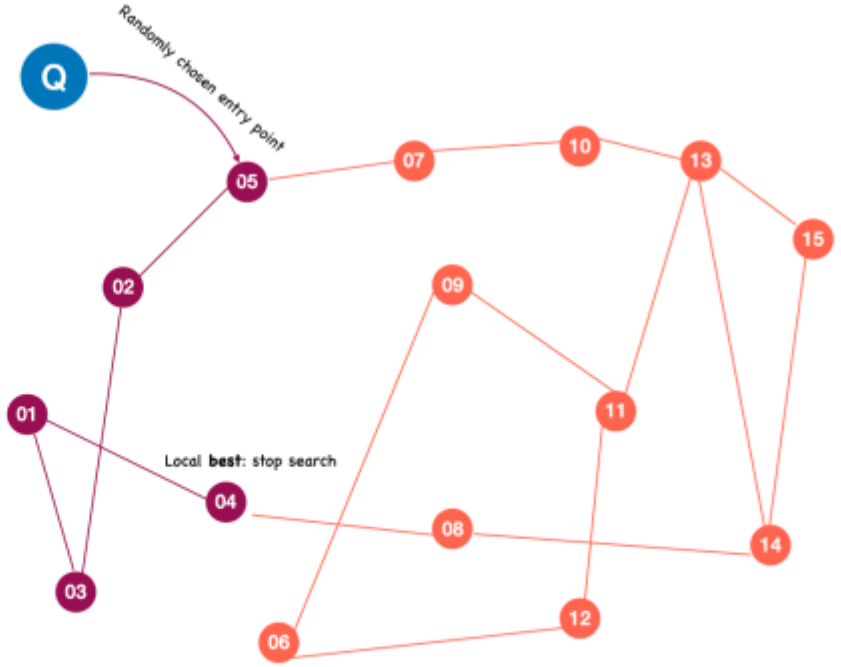




En RAG nos importa principalmente el **recall** — si perdemos un chunk relevante, la respuesta puede ser incorrecta.

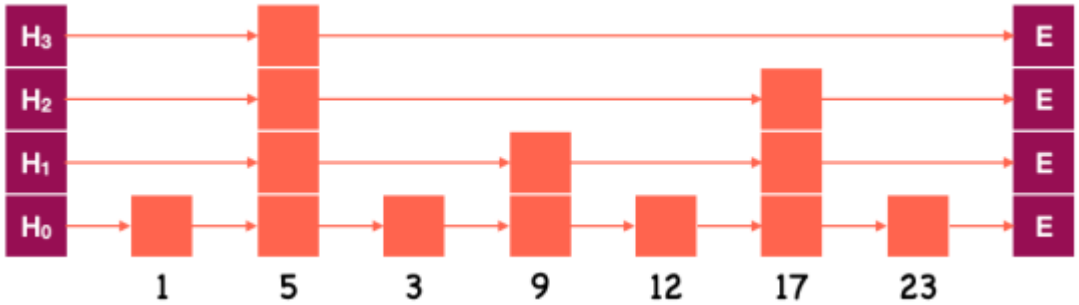
NSW: Navigable Small World

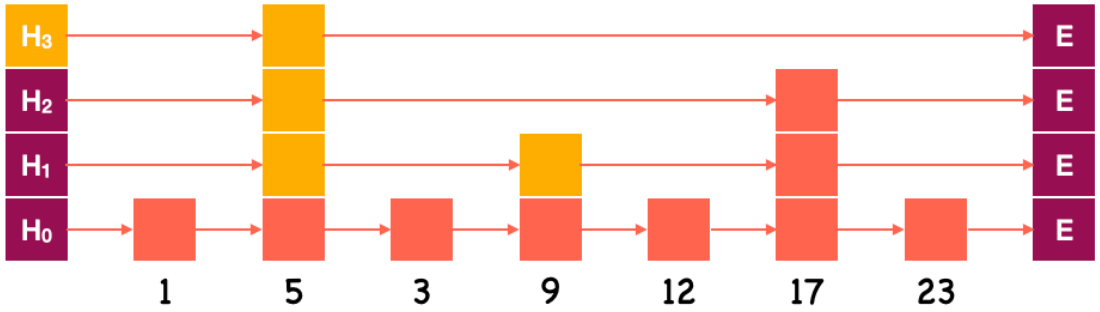
Node	Text
01	The transformer is a model
02	Diagnose tumor with AI
03	A transformer-based model
04	Transformer has 12 layers
05	An MRI costs 10000$
06	The dot product is based on cosine function
07	Big-Pharma is not very big
08	Cross-Attention is a grate attention method
09	To solve an ODE you need math
10	We are aging too fast because of disease
11	Open-Source models are free
12	Bert is a two phase model
13	AI can control aging in future
14	Attention is all you need is of Google
15	LLaMA-2 has & billions parameters

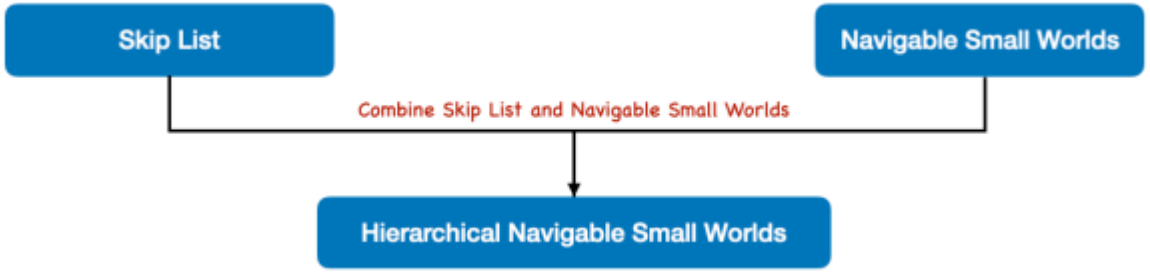


We repeat the search without randomly chosen start points and then keep the top K among all the visited nodes.

HNSW: Hierarchical Navigable Small World

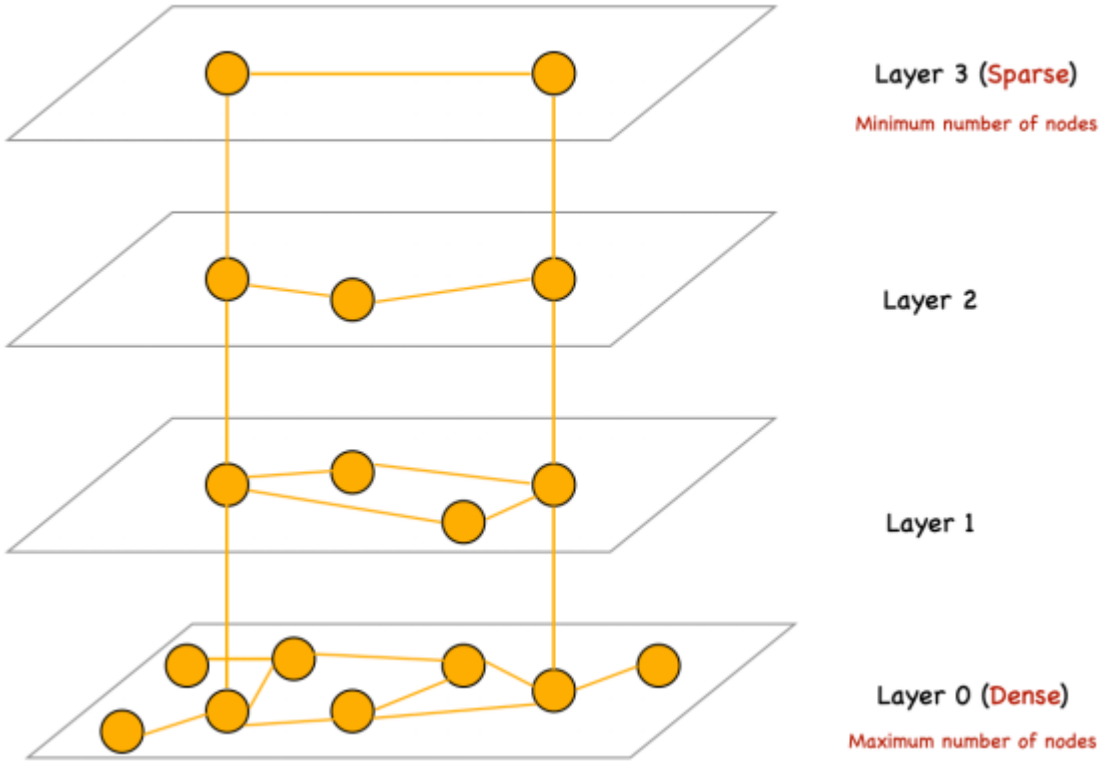


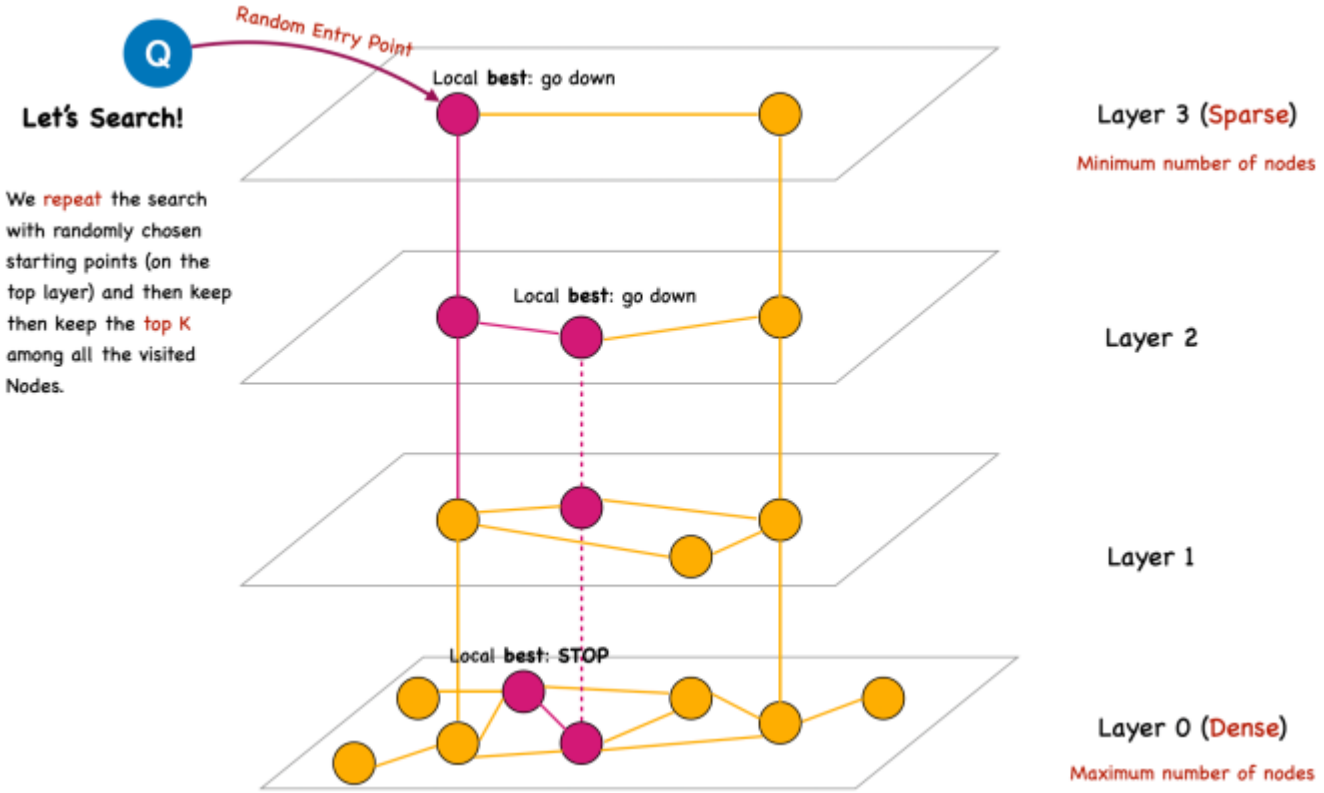
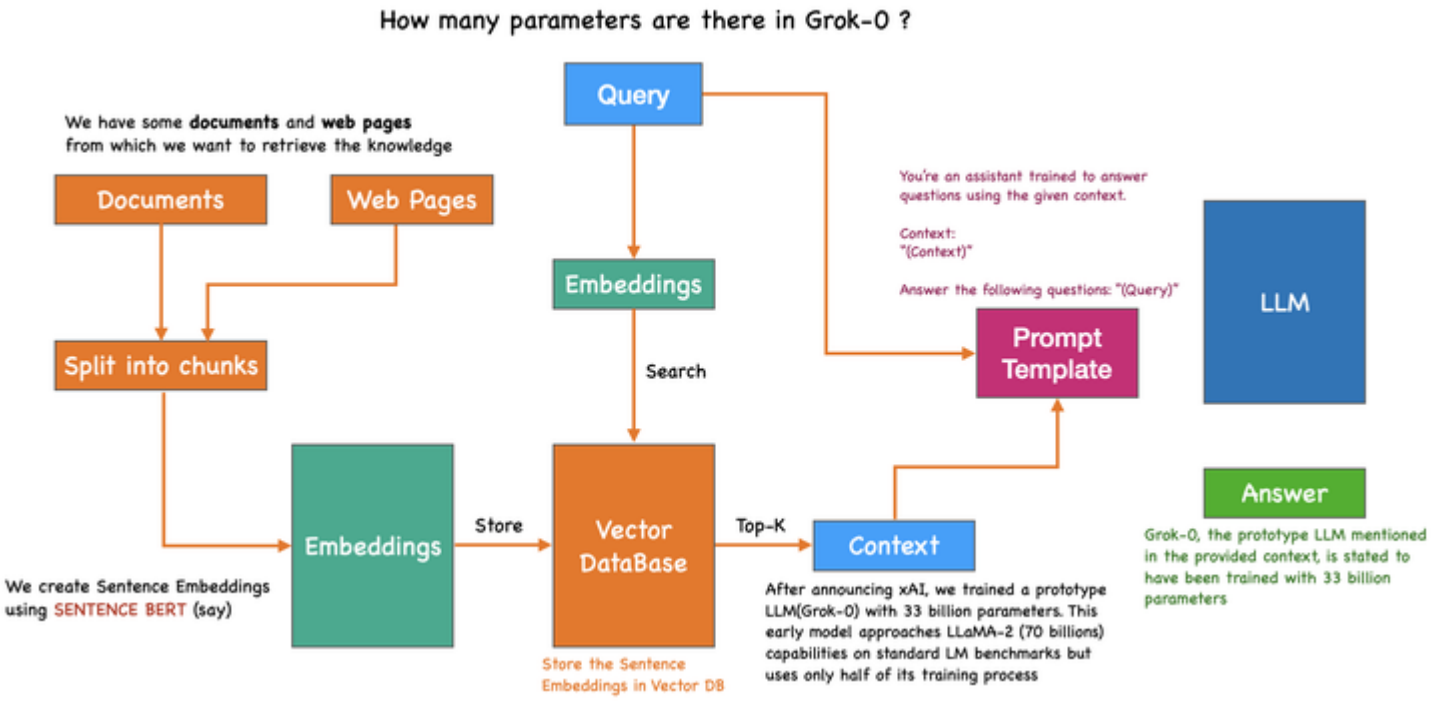




HNSW: Hierarchical Navigable Small World

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería





RAG Pipeline